WEEK 2 · DAY 1 · OVERVIEW

Dates and Coupon Schedules



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

Week 2 moves our four bonds off the coupon date: they settle Monday 30 November 2026. Before anyone prices them, I need three facts per bond: the previous coupon, the next coupon, and how many are left, each tied to Excel. Put the date logic in bondmath, with tests. Then the holdings file: which bonds pay a coupon before year end, and how much cash is that? Last, two benchmark bonds will not tie to Excel: tell me why.

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

Your Route Through Day 1

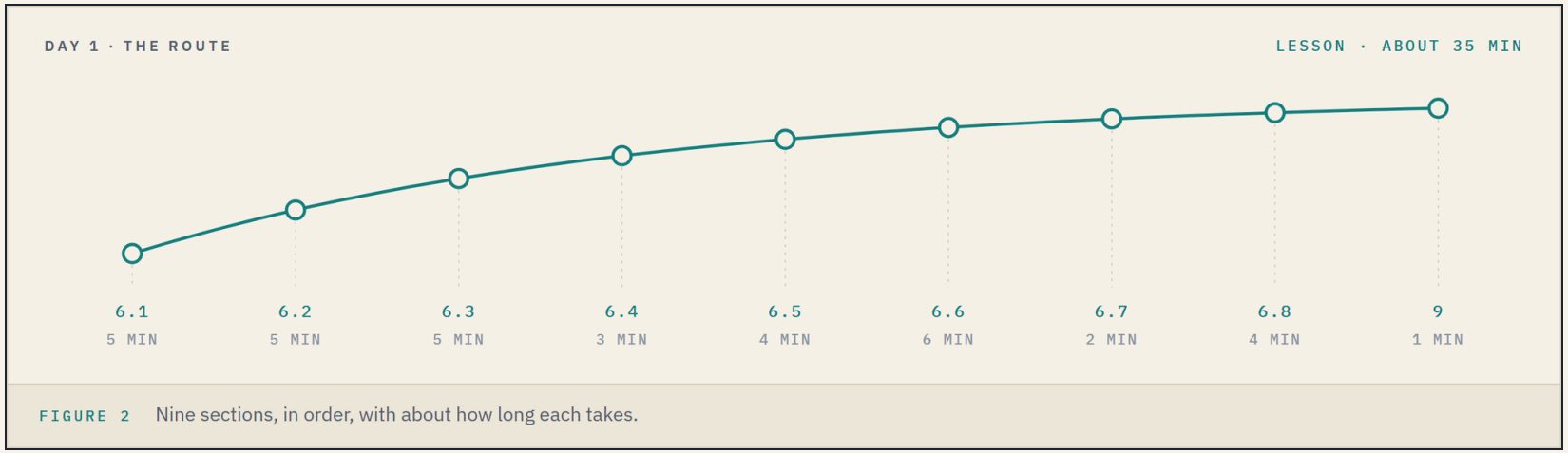


FIGURE 2 Nine sections, in order, with about how long each takes.

	SECTION	ABOUT
6.1	Dates in Python	5 min
6.2	Six Months Is Not 182 Days	5 min
6.3	`add_months`	5 min
6.4	Count Back From Maturity	3 min
6.5	The Month-End Rule	4 min

	SECTION	ABOUT
6.6	Previous, Next, and Coupons Left	6 min
6.7	Keep the Cache Out of Git	2 min
6.8	The Holdings' Coupon Calendar	4 min
9	Bring It Home	1 min

Time. About 35 minutes for the lesson, then about 12 for the four exercises.

6.1 Dates in Python

DAYS BETWEEN TWO DATES

```
issue = date(2026, 9, 30)
settlement = date(2026, 11, 30)
print('Issue to settlement:', (settlement - issue).days, 'days')
```

```
Issue to settlement: 61 days
```

EXCEL'S SERIAL NUMBER

```
excel_serial = (settlement - date(1899, 12, 30)).days
print('Excel serial of', settlement, 'is', excel_serial)
```

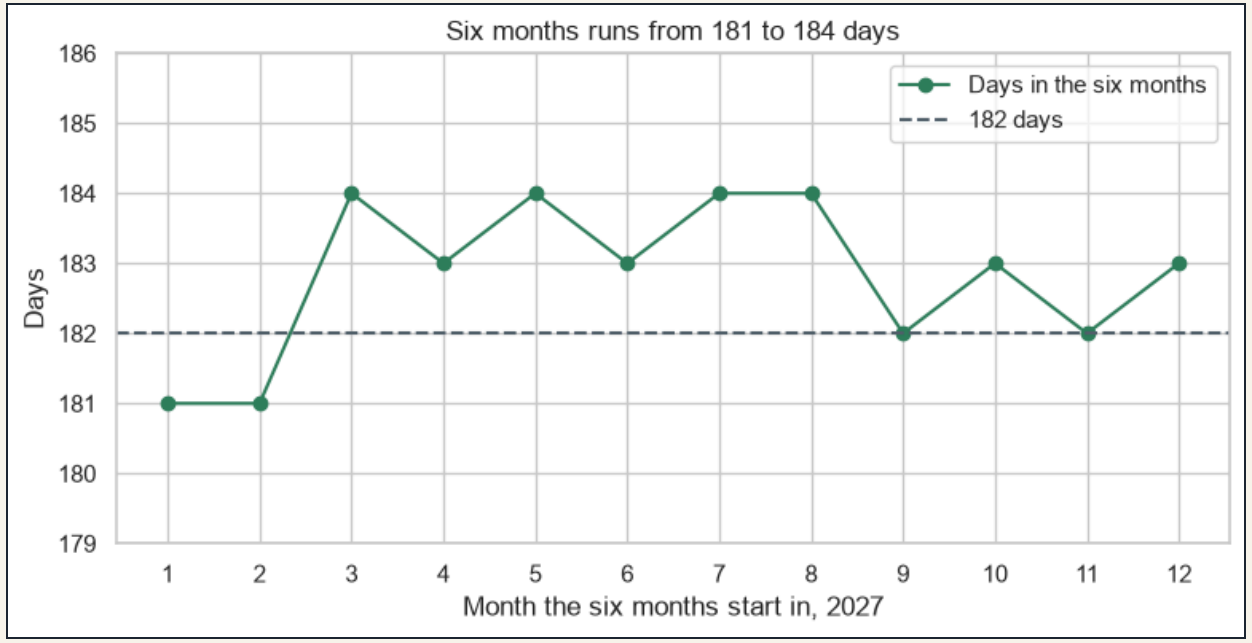
```
Excel serial of 2026-11-30 is 46356
```

EXCEL SIDE BY SIDE

EXCEL	RESULT
`=DATE(2026,11,30)-DATE(2026,9,30)`	61
`=TEXT(DATE(2026,11,30),"dddd")`	Monday
`=YEAR(DATE(2026,11,30)), MONTH, DAY`	2026, 11, 30
`=DATEVALUE("2031-09-30")`	48121

A `date` adds, subtracts, and compares; Excel's dates are day counts, so the two agree.

6.2 Six Months Is Not 182 Days



SIX MONTHS, COUNTED BY MONTH

```
month_days = []
for year in [2027, 2028]:
    for month in range(1, 13):
        month_days.append(calendar.monthrange(year, month)[1])
six_months = [sum(month_days[i:i + 6]) for i in range(12)]
```

```
February 2027: 28  February 2028: 29
[181, 181, 184, 183, 184, 183, 184, 184, 182, 183, 182, 183]
```

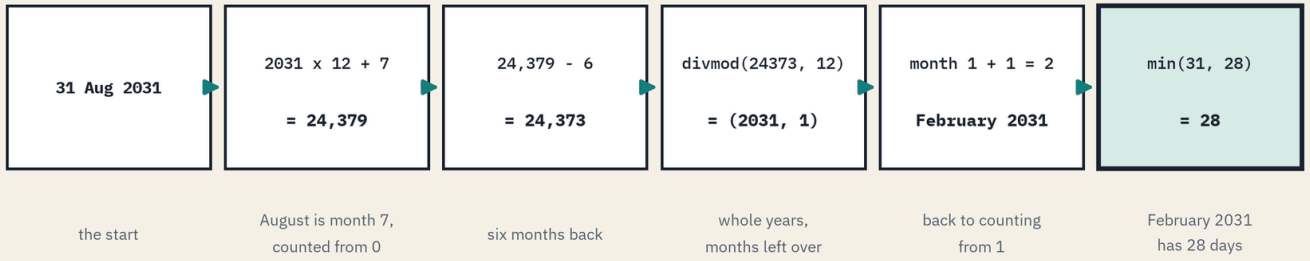
EXCEL SIDE BY SIDE

EXCEL	RESULT
`=DAY(EOMONTH(DATE(2027,2,1),0))`	28 (2028: 29)

Moving by months must carry into the year and cut the day to a short month.

6.3 add_months

Six months back from 31 August 2031, in one long count of months



The answer: 28 February 2031, as Excel's =EDATE(DATE(2031,8,31),-6) gives it.

FIGURE 3 Six months back from 31 August 2031 lands on 28 February 2031 in five steps.

ONE LONG COUNT OF MONTHS

```
month_index = 2031 * 12 + (8 - 1) - 6
year, month_zero_based = divmod(month_index, 12)
print(month_index, '->', year, month_zero_based + 1)
```

```
24373 -> 2031 2
```

FIVE DATES

```
2031-08-31 -6 -> 2031-02-28
2032-08-31 -6 -> 2032-02-29
2027-01-31 1 -> 2027-02-28
2026-11-15 3 -> 2027-02-15
2031-09-30 -6 -> 2031-03-30
```

add_months is Excel's EDATE, tested.

6.4 Count Back From Maturity

THE LAST FOUR COUPONS

```
for k in range(4):
    print(k, bondmath.add_months(quorvane_maturity, -6 * k))
```

```
0 2031-09-30
1 2031-03-30
2 2030-09-30
3 2030-03-30
```

THE SCHEDULE

A semiannual bond pays every six months, and half of 365 is about 182.

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
Stepped	`bondmath.add_months(stepped[-1], -6)`	maturity in A1, `=EDATE(A1,-6)` in A2, filled down
Counted	`bondmath.add_months(august_maturity, -6 * k)`	1 to 4 in C2:C5, `=EDATE($A$1,-6*C2)` in B2, filled down

Count every coupon date from maturity, never from the date before.

6.5 The Month-End Rule

IS IT A MONTH END?

```
for d in [quorvane_maturity, august_maturity, date(2031, 2, 28), date(2032, 2, 28)]:
    print(d, d.day == calendar.monthrange(d.year, d.month)[1])
```

```
2031-09-30 True
2031-08-30 False
2031-02-28 True
2032-02-28 False
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
A month end, then not	`=DATE(2031,9,30)=EOMONTH(DATE(2031,9,30),0)`, then `DATE(2031,8,30)`	TRUE, then FALSE
February, two years	`=DATE(2031,2,28)=EOMONTH(DATE(2031,2,28),0)`, then 2032	TRUE, then FALSE

`EOMONTH(date, months)` **moves a date by whole months and returns that month's last day.**

6.6 Previous, Next, and Coupons Left

bondmath.coupon_dates(settlement, quorvane_maturity): one list, three answers

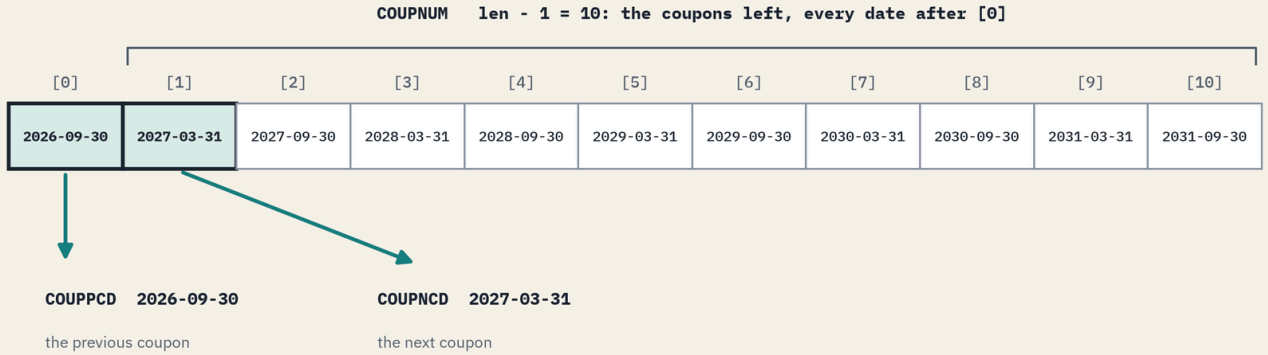


FIGURE 6 Quorvane's eleven dates: item 0 is COUPPCD, item 1 is COUPNCD, and 10 are left.

QUORVANE ON 30 NOVEMBER

```
Previous 2026-09-30  next 2027-03-31  coupons left 10
```

FOUR BONDS AGAINST EXCEL

	TICKER	NEXT	LEFT
0	QVNE	2027-03-31	10
1	DNMR	2027-03-31	14
2	PLWK	2027-03-31	6
3	HLVF	2027-03-31	20

The note's first ask: 30 September and 31 March for all four, and 10, 14, 6, and 20 coupons left, as Excel says on all 306 rows.

6.7 Keep the Cache Out of Git

WHAT CHANGED

```
!git -C "{work}" status --short
```

```
M bondmath.py
 M test_bondmath.py
?? __pycache__/
```

WRITE .GITIGNORE

```
%%writefile .gitignore
__pycache__/
.pytest_cache/
```

```
Writing .gitignore
```

THE RECORD

```
e169ff0 (HEAD -> main) Notebook 06: add_months, coupon_dates, and a .gitignore
322960d (tag: week1) Notebook 05: tests of par pricing and of price against yield
7147a11 Notebook 04: yield_from_price
3f57cac Notebook 03: convert_yield
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

Git records the work, not what Python rebuilds.

6.8 The Holdings' Coupon Calendar



MONTH ENDS IN THE FILE

```
200 bonds, 0 maturing on a month end
```

NEXT COUPON FOR EACH BOND

	CUSIP	NEXT_COUPON
0	99080CAE8	2026-12-18
1	99049XAE2	2026-10-17
2	99039NAC0	2026-12-03

None, so the month-end rule never touches this file.

Two Systems Disagree

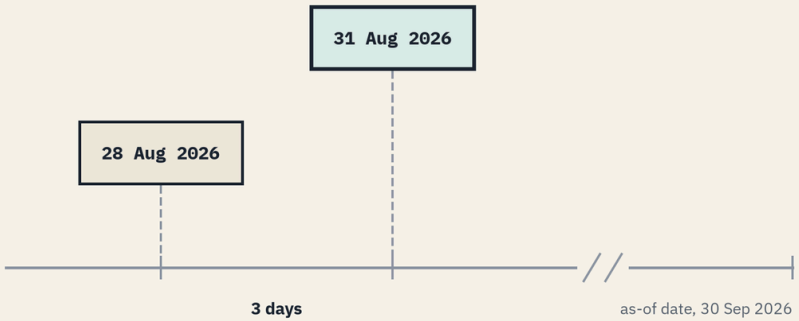
99018SAA8 matures on 28 February 2038, the last day of February: two schedules

Excel's month-end rule

every coupon on a month's last day

The data file

the maturity's day kept: the 28th



Previous coupons three days apart, so accrued differs. The bond's own documents decide.

FIGURE 7 Bond 99018SAA8's previous coupon is 31 August 2026 by Excel, or 28 August 2026 in the file.

BENCHMARK BONDS ON A MONTH END

```
cusip  as_of_date    maturity  accrued
96   99018SAA8  2026-09-30  2038-02-28    0.456
706  99128YAE4  2026-09-30  2035-02-28    0.844
```

TWO FEBRUARY BONDS

```
99018SAA8, month-end rule: 2026-08-31
99018SAA8, day kept: 2026-08-28
```

113 bonds pay 8,540,781.25 dollars of coupon by year end; the two February bonds sit three days apart by convention.

Bring It Home

IN A POWERSHELL TERMINAL IN VS CODE

```
Set-Location "$HOME\projects\my_bondmath"
$work = "$env:TEMP\pfi_course2_06_k4w7q2xr"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py", "$work\.gitignore" .
python -m pytest -q
git status --short
git add bondmath.py test_bondmath.py .gitignore
git commit -m "Notebook 06: add_months, coupon_dates, and a .gitignore"
```

The copy replaces your files, so anything of your own goes into the work folder first.

You Can Now

- Make, compare, and subtract dates, and say what Excel's date serials count.
- Move a date by months with **`add_months`** or **`EDATE`**.
- Count coupon dates back from maturity, and say why stepping drifts.
- Apply the month-end rule, with **`EOMONTH`** in Excel.
- Find the previous coupon, next coupon, and coupons left with **`coupon_dates`**, tied to Excel.
- Hand a pandas date column to the module with **`.dt.date`**.
- Keep the cache out of Git with a **`.gitignore`**.

NEXT
WEEK 2 · DAY 2

Day Counts and Accrued Interest

What a buyer owes the seller between coupons: 30/360 and actual/actual, the day counts A, E, and DSC, and three functions tied to Excel and to the holdings file.